

2026 암호분석경진대회

3번 문제

위조된 TLS record는 다음과 같다.

0x17030300489c269a9f29810ab0a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc36a29772edbf159643b468f1f1e520e7341daee867a49665542cd9f01f5a9c906b73ba3e13bbfea0e98a18

1. Notation

P	평문
C	암호문
P_i	i번째 128-bit 평문, $P = P_1 P_2 \dots P_n$
C_i	i번째 128-bit 암호문, $C = C_1 C_2 \dots C_n$
$P^{(i)}$	i번째 Application Data의 평문
$C^{(i)}$	i번째 Application Data의 암호문
K	문제에서 서버와 클라이언트가 공유하고 있는 128-bit AES 비밀키
E_K	비밀키 K 을 통해 128-bit 평문을 AES-128로 암호화하는 함수
$A B$	bit concatenation
$A \cdot B$	문제 풀이에서 다루는 곱셈은 모두 $GF(2^{128})$ 위에서 이루어진다. A 와 B 모두 $GF(2^{128})$ 이며 사용되는 기약다항식은 $x^{128} + x^7 + x^2 + x + 1$ 을 사용한다.
$PAD(A)$	128-bit zero 패딩, A 가 120-bit일 경우 8bit는 0으로 채운다.
AD_i	AES_128_GCM Authentication Tag 생성에 사용되는 인증된 데이터로, 본 문제에서는 아래와 같이 초기화된다. $AD_i = \text{seq_num}(8\text{-byte}) \parallel i \parallel 0x170303 \parallel C^{(i)}$ 의 byte 길이(2-byte) (seq_num의 경우, AES_128_GCM 암호화 통신 시작부터 1씩 증가하는 값이다. 아래 Application Data에서, seq_num 값은 순서대로 1, 2, 3, 4이다.)

2. 취약점 분석

1) TLS 패킷 분석

패킷 Info	내용
Client Hello	TLS 1.2 프로토콜로 통신을 수행한다. cipher suite로 0xC02F를 서버에 제안한다.
Server Hello	클라이언트가 제안한 cipher suite인 0xC02F를 선택함으로써, TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 방식으로 암호화 통신을 진행한다.
Certificate ~ Change Cipher Spec	서버는 X.509 디지털 인증서를 클라이언트에게 보내 신원 증명을 수행하고 이후 ECDHE 방식을 통해 키 교환을 진행하며, 키 교환이 완료되면 AES-GCM 방식을 통해 암호화 통신을 수행한다.
Application Data	클라이언트(본사 서버)가 서버(각 지사)로 급여 정보를 전달하며, 해당 메시지들은 AES_128_GCM 방식으로 암호화되어 전달된다.

2) Application Data의 TLS record 분석

총 4개의 Application Data가 서버로 전송되며 각 Data의 TLS record는 다음과 같다.

0x17 0303 003a 9c269a9f29810ab0 a99141d1d84c3df5b3c6e328f879f64b2bcd6561fe7fe19a73edad45cd0fb968f4f0 035dd4f7202b87c66b7a2f2442f6d53f
0x17 0303 004a 9c269a9f29810ab0 a99141d1d84c3df4afcdf677d263f31a7a8f2c70ad62f69a72eaa943dd56e42cf9f1e436ba7d1eb5ee2efe9b65522edc6860 99935ebb98284af75a6253d1b5c10ec1
0x17 0303 0048 9c269a9f29810ab0 a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc36a29772edbf159643b468f5f1e520e7341daee867a49665542cd9 b0d6e0ac2be205621f7ea4a034519f95

0x17||0303||0040||9c269a9f29810ab1||59fa2fa8e2b59bcfac98818ad5125321fca496d17d6a8d7b35dc795b9248e1a1156b47aa4654aa98||a2197f30dea94e1565e1f28f540268ac

이때, TLS record는 다음과 같은 구조로 이루어져있다.

Content Type || TLS Version || Length || Explicit Nonce || Ciphertext || Authentication Tag

이를 통해 1, 2, 3번째 Application Data에서 동일한 Explicit Nonce 값을 사용했음을 알 수 있다.

3) AES_128_GCM 구조

AES_128_GCM에서 사용되는 Nonce는 12-byte 값으로, 4-byte 크기의 Implicit Nonce와 8-byte 크기의 Explicit Nonce로 이루어진다(Nonce = Implicit Nonce||Explicit Nonce). 해당 Nonce 값은 96-bit이므로, 128-bit인 Counter_0 값은 다음과 같이 초기화된다.

$$\text{Counter}_0 = \text{Nonce}||0\dots0||1$$

이후 순차적으로 생성되는 각 Counter_1, Counter_2, ... 값은 이전 카운터에서 1씩 증가한 값이다. Implicit Nonce는 핸드셰이크 과정에서 클라이언트와 서버가 공유한 고정값이므로 각 TLS record의 Explicit Nonce가 동일할 경우, 해당 TLS record의 AES_128_GCM 암호화에서 사용된 Nonce 값은 동일한 값이다.

AES_128_GCM은 AEAD 암호화 방식 중 하나로, 암호문과 Authentication Tag를 생성한다. 암호문은 다음과 같이 생성되며

$$C_i = P_i \oplus E_K(\text{Counter}_i)$$

Authentication Tag는 다음과 같이 생성된다.

$$\begin{aligned}
 H &= E_K(0) \\
 S_i &= \begin{cases} AD_i & \text{for } i = 1, \dots, m-1 \\ PAD(AD_m) & \text{for } i = m \\ C_{i-m} & \text{for } i = m+1, \dots, m+n-1 \\ PAD(C_n) & \text{for } i = m+n \\ len(A)||len(C) & \text{for } i = m+n+1 \end{cases} \\
 X_i &= \begin{cases} 0 & i = 0 \\ (X_{i-1} \oplus S_i) \cdot H & i = 1, \dots, m+n+1 \end{cases} \\
 \text{Authentication Tag} &= X_{m+n+1} \oplus E_K(\text{Counter}_0) = GHASH(H, AD, C) \oplus E_K(\text{Counter}_0)
 \end{aligned}$$

이때 $len(A)$ 는 A 의 비트 길이의 64-bit 표현이다.

4) AES_128_GCM Nonce 재사용 취약점

본 문제와 같이 3개의 Application Data에서 Nonce를 재사용하였을 경우 다음과 같이 Authentication Tag를 위조할 수 있다. i 번째 Application Data의 Authentication Tag를 AT_i 라 할 때, AT_i 는 다음과 같다.

$$AT_i = GHASH(H, AD_i, C^{(i)}) \oplus E_K(\text{Counter}_0)$$

Authentication Tag를 위조하려면 H 와 $E_K(\text{Counter}_0)$ 의 값을 알아야한다. 이때, 3개의 Application Data는 동일한 Nonce를 사용하였으므로 해당 데이터들의 $E_K(\text{Counter}_0)$ 값은 같다. 따라서 다음과 같은 3개의 다항식을 고려하여,

$$f_1(z) = GHASH(z, AD_1, C^{(1)}) \oplus AT_1$$

$$f_2(z) = GHASH(z, AD_2, C^{(2)}) \oplus AT_2$$

$$f_3(z) = GHASH(z, AD_3, C^{(3)}) \oplus AT_3$$

$\Delta f_1 = f_1 \oplus f_2$, $\Delta f_2 = f_2 \oplus f_3$ 을 계산할 경우, H 를 근으로 갖는 서로 다른 두 다항식 $\Delta f_1, \Delta f_2$ 을 얻을 수 있다.

즉 높은 확률로 $GCD(\Delta f_1, \Delta f_2)$ 을 계산함으로써 $(z - H)$, H 를 얻을 수 있다. 이후 $GHASH(H, AD_3, C^{(3)})$ 값을 구하고, $AT_3 \oplus GHASH(H, AD_3, C^{(3)})$ 을 계산하여 $E_K(\text{Counter}_0)$ 값을 쉽게 얻을 수 있다.

3. 암호문 위조

3번째 Application data의 평문은 아래와 같은 형식을 따름을 알고 있으므로,

$action=<cmd>\&uid=<4-digit>\&amt=<4-digit>\&month=<6-digit>$

문제 예시에서 제시한 바이트 문자열을 예상 평문으로 사용하여 48-byte KEY_STREAM 을 얻어낸다.

$P^{(3)} = \text{b'action=set_salary\&uid=0007\&amt=0100\&month=202603'}$	
$KEY_STREAM = C^{(3)} \oplus P^{(3)}$	
KEY_STREAM	0xc8f235b8b7220085daa8840e8d1092761bfd5556d80b92a742da9974fb378958c4c1d5068a5b73da805a96a657621cea

3개의 Application data는 동일한 Nonce 값을 사용하였으므로 해당 KEY_STREAM 은 1번째, 2번째 Application Data 내용의 복호화에도 사용될 수 있다. 복호화한 결과는 다음과 같다.

$P^{(1)} = \text{b'action=ping\&uid=0007\&ts=1741680001'}$
$P^{(2)} = \text{b'action=query_salary\&uid=0007\&amt=0100\&month=2026'}$
$(P^{(2)})$ 의 길이는 50-byte이므로 완벽히 복호화하기 위해서는 2-byte의 추가 KEY_STREAM 이 필요하다. 그러나 $P^{(3)}$ 와 application data의 형식을 참고할 때 아래와 같이 완전한 $P^{(2)}$ 의 값을 추측할 수 있다. $P^{(2)} = \text{b'action=query_salary\&uid=0007\&amt=0100\&month=202603'}$

이를 통해 예상 평문 $P^{(3)}$ 가 실제 평문이었음을 알 수 있다. 이제 위조 암호문을 생성하기 위해 다음과 같은 위조 평문 $FP^{(3)}$ 을 준비한 뒤, 위조 암호문 $FC^{(3)}$ 을 구한다.

$FP^{(3)} = \text{b'action=set_salary\&uid=0007\&amt=0500\&month=202603'}$
$FC^{(3)} = KEY_STREAM \oplus FP^{(3)}$

이제 위조된 Authentication Tag를 구하기 위해 H 와 $E_K(\text{Counter}_0)$ 값을 구해야 한다. 이를 위해, 다음과 같이 $GF(2^{128})$ 의 원소들을 계수로 갖는 다항식을 고려한다.

$$f_1(z) = 0 \cdot z^6 \oplus PAD(AD_1)z^5 \oplus C_1^{(1)}z^4 \oplus C_2^{(1)}z^3 \oplus PAD(C_3^{(1)})z^2 \oplus (len(AD_1)||len(C^{(1)}))z \oplus AT_1$$

$$f_2(z) = PAD(AD_2)z^6 \oplus C_1^{(2)}z^5 \oplus C_2^{(2)}z^4 \oplus C_3^{(2)}z^3 \oplus PAD(C_4^{(2)})z^2 \oplus (len(AD_2)||len(C^{(2)}))z \oplus AT_2$$

$$f_3(z) = 0 \cdot z^6 \oplus PAD(AD_3)z^5 \oplus C_1^{(3)}z^4 \oplus C_2^{(3)}z^3 \oplus C_3^{(3)}z^2 \oplus (len(AD_3)||len(C^{(3)}))z \oplus AT_3$$

$f_i(H) = GHASH(H, AD_i, C^{(i)}) \oplus AT_i = E_K(\text{Counter}_0)$ 이다. 이제 두 다항식의 차분 $\Delta f_1, \Delta f_2$ 을 고려하자.

$$\Delta f_1 = f_1 \oplus f_2$$

$$\Delta f_2 = f_2 \oplus f_3$$

이때 $\Delta f_1(H) = 0, \Delta f_2(H) = 0$ 이 성립한다. 즉 $\Delta f_1, \Delta f_2$ 는 $(z - H)$ 을 공통 인자로 가지므로, $GCD(\Delta f_1, \Delta f_2)$ 을 계산함으로써 $(z - H)$, H 을 얻을 수 있다. 이후 $AT_3 \oplus GHASH(H, AD_3, C^{(3)})$ 을 계산하여 $E_K(\text{Counter}_0)$ 을 얻어낼 수 있다. 얻어낸 H 와 $E_K(\text{Counter}_0)$ 의 값은 다음과 같다.

H	0x112332a84132bc0c5c23a61037723683
$E_K(\text{Counter}_0)$	0x1e023d8461003896981f1441c5feab90

최종적으로 위조된 TLS record는 다음과 같이 생성한다.

0x17 0303 0048 9c269a9f29810ab0 $FC^{(3)} GHASH(H, AD_3, FC^{(3)}) \oplus E_K(\text{Counter}_0)$
--

4. 코드 구현

본 문제의 공격 코드는 sage를 통해 구현되었다. solve.sage와 assist.sage 2개의 파일이 있으며, solve.sage를 실행하여 위조된 TLS record 문자열을 얻을 수 있다.

assist.sage에 있는 주요 함수들의 명세는 다음과 같다.

block2K	bytes 값을 $GF(2^{128})$ 의 원소로 변환
K2block	$GF(2^{128})$ 의 원소를 bytes 값으로 변환
ghash_blocks	AD 와 C 을 인자로 받아 f_i 의 계수들을 list 형태로 반환
diff_poly	Δf_i 을 계산

solve.sage 실행 결과

```
$ sage solve.sage
[+] Successfully Obtained H : 112332a84132bc0c5c23a61037723683
P^(1) : b'action=ping&uid=0007&ts=1741680001'
P^(2) : b'action=query_salary&uid=0007&amt=0100&month=2026'
Key_stream :
c8f235b8b7220085daa8840e8d1092761bfd5556d80b92a742da9974fb378958c4c1d5068a5b73da805a96a6576
21cea
[+] Successfully Obtained E_K(counter0) : 1e023d8461003896981f1441c5feab90

#E_K(Counter_0)와 H가 제대로 구해졌는지, AT_1, AT_2을 계산 후 비교하는 테스트 수행
[+] ==== Test 1 result ====
[+] Result : True
[+] Test expect : b'\xb0\xd6\xe0\xac+\xe2\x05b\x1f~\xa4\xa04Q\x9f\x95'
[+] Test original : b'\xb0\xd6\xe0\xac+\xe2\x05b\x1f~\xa4\xa04Q\x9f\x95'

[+] ==== Test 2 result ====
[+] Result : True
[+] Test expect : b'\x99\x93^\xbb\x98(J\x7fZbS\xd1\xb5\xc1\x0e\xc1'
[+] Test original : b'\x99\x93^\xbb\x98(J\x7fZbS\xd1\xb5\xc1\x0e\xc1'

[+] ==== Final Result ====
[+] Forge_message_hex :
17030300489c269a9f29810ab0a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc36a29772edbf159643b468f1f
1e520e7341daee867a49665542cd9f01f5a9c906b73ba3e13bbfea0e98a18
```